

TASK 3 – DBMS

Seller and Inventory Management System

1. Aim

To design and implement a MySQL-based Seller and Inventory Management System that helps maintain supplier details, product records, stock quantities, pricing information, and inventory status.

2. Objectives

- * Create a database for managing seller and product information.
- * Store seller contact and business details.
- * Maintain product name, category, price, and seller information.
- * Track the quantity of products available in the warehouse.
- * Identify products with low, sufficient, or unavailable stock.
- * Establish relationships between sellers, products, and inventory.
- * Perform INSERT, UPDATE, DELETE, and SELECT operations.
- * Generate useful inventory summaries using SQL queries.

3. Database Design

The system consists of three main tables:

Seller Table

- * Seller ID
- * Seller Name
- * Business Name
- * Phone Number
- * Email

Product Table

- * Product ID
- * Product Name
- * Category
- * Price
- * Seller ID

Inventory Table

- * Inventory ID
- * Product ID
- * Quantity
- * Reorder Level
- * Stock Status

Relationship

****Seller → Product → Inventory****

One seller can provide many products. Each product has a corresponding inventory record.

4. Create Database

```
CREATE DATABASE shop_inventory_db;
```

```
USE shop_inventory_db;
```

5. Create Seller Table

```
CREATE TABLE Seller (  
    seller_id INT PRIMARY KEY,  
    seller_name VARCHAR(50),  
    business_name VARCHAR(60),  
    phone VARCHAR(15),  
    email VARCHAR(60)  
)
```

6. Create Product Table

```
CREATE TABLE Product (  
    product_id INT PRIMARY KEY,  
    product_name VARCHAR(50),  
    category VARCHAR(40),  
    price DECIMAL(10,2),  
    seller_id INT,  
    FOREIGN KEY (seller_id) REFERENCES Seller(seller_id)  
)
```

7. Create Inventory Table

```
CREATE TABLE Inventory (  
    inventory_id INT PRIMARY KEY,  
    product_id INT,  
    quantity INT,  
    reorder_level INT,  
    stock_status VARCHAR(20),  
    FOREIGN KEY (product_id) REFERENCES Product(product_id)  
)
```

8. Insert Seller Records

```
INSERT INTO Seller VALUES  
(1, 'Karthik Stores', 'Karthik Electronics', '9876501234', 'karthik@shop.com'),  
(2, 'Priya Traders', 'Priya Digital Mart', '9867401235', 'priya@shop.com'),  
(3, 'Ravi Enterprises', 'Ravi Computer World', '9856301246', 'ravi@shop.com');
```

9. Insert Product Records

```
INSERT INTO Product VALUES  
(301, 'Keyboard', 'Computer Accessories', 750.00, 1),  
(302, 'Mouse', 'Computer Accessories', 450.00, 2),  
(303, 'Bluetooth Speaker', 'Audio', 2200.00, 3),  
(304, 'USB Drive', 'Storage', 650.00, 1);
```

10. Insert Inventory Records

```
INSERT INTO Inventory VALUES  
(21, 301, 25, 10, 'Available'),  
(22, 302, 7, 10, 'Low Stock'),  
(23, 303, 15, 5, 'Available'),  
(24, 304, 0, 5, 'Out of Stock');
```

11. Display Seller Information

```
SELECT * FROM Seller;
```

This query displays all seller details stored in the Seller table.

12. Display Product Information

```
SELECT * FROM Product;
```

This query displays product ID, product name, category, price, and seller ID.

13. Display Inventory Information

```
SELECT * FROM Inventory;
```

This query displays the current stock quantity and stock status of each product.

14. Display Product and Seller Details

```
SELECT
```

```
    Seller.business_name,  
    Product.product_name,  
    Product.category,  
    Product.price
```

```
FROM Seller
```

```
INNER JOIN Product
```

```
ON Seller.seller_id = Product.seller_id;
```

This query combines seller and product information using an INNER JOIN.

15. Display Products with Available Stock

```
SELECT
```

```
    Product.product_name,  
    Inventory.quantity,  
    Inventory.stock_status
```

```
FROM Product
```

```
INNER JOIN Inventory
```

```
ON Product.product_id = Inventory.product_id
```

```
WHERE Inventory.stock_status = 'Available';
```

16. Display Low Stock Products

```
SELECT
    Product.product_name,
    Inventory.quantity,
    Inventory.reorder_level
FROM Product
INNER JOIN Inventory
ON Product.product_id = Inventory.product_id
WHERE Inventory.stock_status = 'Low Stock';
```

17. Generate Complete Inventory Report

```
SELECT
    Product.product_id,
    Product.product_name,
    Product.category,
    Inventory.quantity,
    Inventory.reorder_level,
    Inventory.stock_status
FROM Product
INNER JOIN Inventory
ON Product.product_id = Inventory.product_id;
```

18. Update Inventory Quantity

For example, if the keyboard stock is increased:

```
UPDATE Inventory
SET quantity = 35,
    stock_status = 'Available'
WHERE product_id = 301;
```

The inventory quantity is updated from 25 to 35.

19. Delete a Product Record

Before deleting the product, its related inventory record is removed.

```
```sql
```

```
DELETE FROM Inventory
```

```
WHERE product_id = 304;
```

```
DELETE FROM Product
```

```
WHERE product_id = 304;
```

## 20. Sample Output

Product Name	Category	Quantity	Stock Status
-----	-----	-----	-----
Keyboard	Computer Accessories	25	Available
Mouse	Computer Accessories	7	Low Stock
Bluetooth Speaker	Audio	15	Available
USB Drive	Storage	0	Out of Stock

## 21. Conclusion

The Seller and Inventory Management System provides an efficient way to organize seller, product, and stock information. The database uses primary keys and foreign keys to maintain relationships between different tables. SQL commands such as insertion, updating, deletion, filtering, and joining help manage the database effectively. The system also makes it easier to identify available, low-stock, and out-of-stock products and supports the generation of useful inventory reports.